Relatório — Loja de Jogos (catálogo de produtos em Flutter)

Aplicativo: Loja de Jogos (catalogo_n2)
Integrantes: [preencher nomes e RA dos integrantes]

Observação sobre o ambiente de teste. Estas capturas foram geradas executando o *mesmo código* entregue como Flutter Web em um Chromium (resolução de celular, 412x860), pois o ambiente em que o projeto foi desenvolvido não possui emulador Android. Os testes de segundo plano com o botão Home do Android (itens 3 e 4 do enunciado) **precisam ser executados e registrados pelo grupo** em um dispositivo/emulador; a seção 6 indica onde inserir essa evidência.

1. O que foi implementado

Catálogo de jogos de tabuleiro com quatro produtos (Catan, Ticket to Ride, Dixit e Carcassonne). Cada card mostra nome, preço, um ícone com o indicador de quantidade sobreposto e o botão *Detalhes*. Há uma segunda tela com os dados completos do produto, uma tela *Transformações* e um histórico do ciclo de vida exibido no catálogo e nos detalhes.

2. Catálogo e indicadores de quantidade

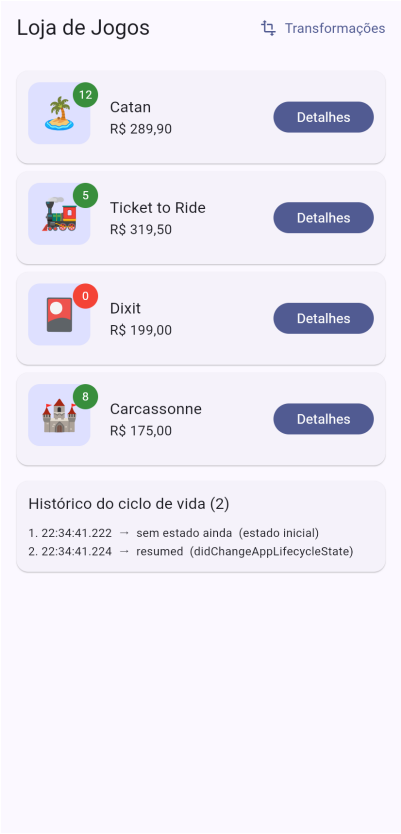
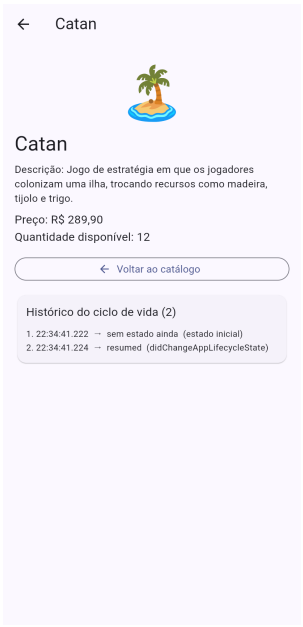
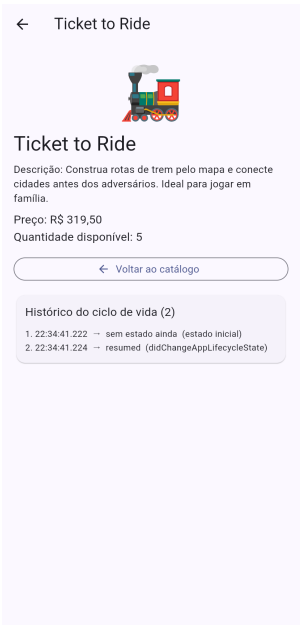


Figura 1 — Catálogo gerado com map()/toList(). O círculo no canto de cada ícone é o indicador de quantidade (verde: em estoque; vermelho: Dixit com 0 unidades).

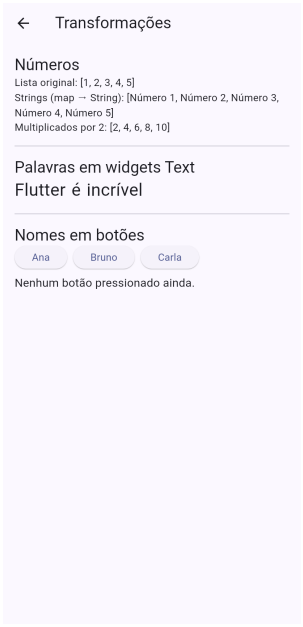
3. Detalhes e navegação





Figuras 2 e 3 — Detalhes de Catan (R$ 289,90, 12 un.) e Ticket to Ride (R$ 319,50, 5 un.). Cada botão carrega o objeto Produto do seu próprio card, por isso as telas mostram dados diferentes. A navegação foi verificada com Catan e Ticket to Ride na execução manual e com um teste de widget (flutter test); o mesmo código serve aos demais produtos.

4. Transformações de listas



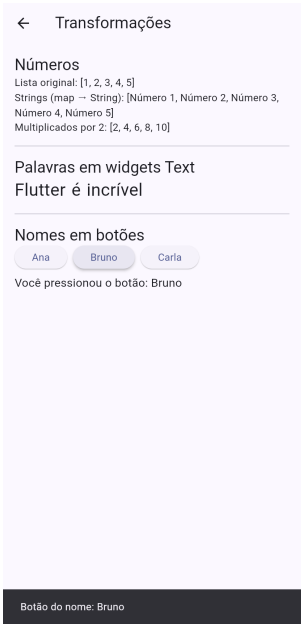


Figura 4 (esq.) — Lista original [1, 2, 3, 4, 5]; lista de strings ("Número 1", ...) obtida com map((n) => 'Número $n').toList(); lista dobrada [2, 4, 6, 8, 10]; as palavras "Flutter", "é", "incrível" convertidas em widgets Text; e os botões Ana, Bruno e Carla. Figura 5 (dir.) — Após tocar em "Bruno", o texto da tela e um SnackBar identificam o nome do botão.

5. Explicações

Classe Produto. Reúne nome, descrição, preço e quantidade (além de um emoji usado como imagem) em campos finais e construtor const. A lista `produtos` contém os objetos; telas leem apenas atributos do objeto, sem depender de listas paralelas.

Envio à tela de detalhes. O botão de cada card é criado dentro do widget daquele produto e passa o próprio objeto no construtor: `DetalhesScreen(produto: produto)`. Como cada card guarda sua referência, o botão identifica corretamente o item.

```
Navigator.push(context, MaterialPageRoute(
  builder: (_) => DetalhesScreen(produto: produto, log: log)));
// ... na tela de detalhes:
Navigator.pop(context);
```

push() e pop(). push() empilha a nova rota sobre o catálogo (que continua na memória, apenas coberto); pop() remove a rota do topo e revela o catálogo novamente. O botão "Voltar ao catálogo" e a seta da AppBar chamam pop().

Stack. Sobrepõe o indicador de quantidade (CircleAvatar em um Positioned) ao quadrado com o ícone do produto, criando um "selo" sem alterar o layout do restante do card.

map() e toList(). map() transforma cada elemento de uma coleção em outro valor, de forma preguiçosa (retorna um Iterable); toList() materializa o resultado em List. Aparecem: (a) catálogo — produtos.map(...)→cards; (b) números→strings; (c) números→dobro; (d) palavras→Text; (e) nomes→ElevatedButton.

6. Ciclo de vida

O histórico é mantido por um LifecycleLog (WidgetsBindingObserver + ChangeNotifier), criado uma única vez em main(); por isso persiste ao navegar entre catálogo e detalhes (Figuras 2 e 3 mostram o mesmo histórico do catálogo). Nenhum estado é adicionado manualmente: cada linha vem de didChangeAppLifecycleState, exceto a linha "estado inicial", que apenas lê WidgetsBinding.instance.lifecycleState quando o observador é registrado.

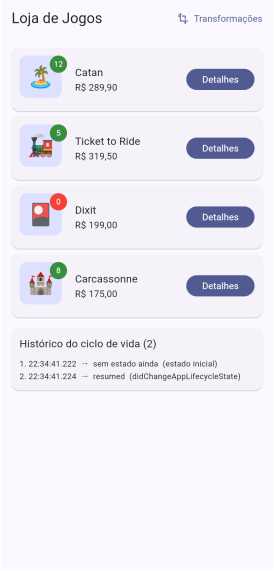


Figura 6 — Histórico observado na execução web: (1) estado inicial (o framework ainda não havia informado estado) e (2) resumed. Nesse ambiente, abrir e fechar os detalhes **não gerou nenhum novo registro**. A troca de aba do navegador automatizado também não gerou eventos, portanto não há evidência de paused nesta execução.

[A COMPLETAR PELO GRUPO — Android] Inserir aqui a captura do histórico após: abrir o app; abrir/fechar detalhes; apertar Home; voltar ao app. Em Android, o esperado é ver algo como inactive

→ hidden → paused ao sair e hidden → inactive → resumed ao retornar, mas o que vale é o que aparecer na execução real de vocês; registrem exatamente isso.

Navegar dentro do app x ir ao segundo plano. Abrir a tela de detalhes é uma mudança interna do Navigator: o app continua visível, em primeiro plano e recebendo eventos, só mudou a rota exibida. Por isso o estado do aplicativo continua resumed e nada é registrado (como observado). Já o botão Home faz o sistema operacional tirar o app da tela: a Activity deixa de ser visível e o Flutter é notificado (inactive/hidden/paused). Paused significa "não visível e sem interação com o usuário", o que não ocorre ao empilhar uma rota dentro do mesmo app.